

CPTS 489

Campus Closet

Final Project Submission

Team Members

Matthew Barnes

Zuriel Hernandez Guerra

Jeremiah Miguel

1. Project Overview

Campus Closet is a web based marketplace for students and alumni to buy and sell collegiate clothing and memorabilia. The platform is designed around university communities rather than general secondhand commerce, so every listing is anchored to a school and every buyer can tune their default feed to the campus they care about most.

Three roles share the product. Buyers browse and search the marketplace, save items to their wishlist, add items to a persisted cart and complete purchases through a local checkout. Sellers inherit all buyer privileges and also manage their own listings, upload a photo for each listing, keep drafts for work in progress and mark items as sold once a transaction completes. Administrators review users and listings, suspend or ban accounts with a recorded reason and remove or restore listings with an admin note attached. The final product is scoped to be demonstrable end to end on a local machine. It is not a production deployment and was never meant to become one.

2. User Roles

Buyer

- Browse listings on the Dashboard and the Shop page.
- View listing details on the Product Detail page.
- Save listings to the Liked Items section on the Dashboard.
- Add items to the cart through the slide in the Cart Sidebar.
- Complete purchases through the local checkout flow.
- Review past orders on the Purchase History page.
- Pick a default university and update it at any time.
- Read account and wishlist notifications from the Notifications page.

Seller

- Retain every buyer capability.
- Open the Seller Dashboard at /seller to see revenue, active listings, items sold and draft listings.
- Create a listing from scratch with a title, category, school, price, size, condition, description and a photo.
- Save a listing as a draft and publish it later.
- Edit any listing that is not already sold or removed.
- Mark an active listing as sold without going through the cart.
- Hide or restore an active listing.
- Receive notifications when an item sells or when an admin hides or removes one of their listings.

Admin

- Access the Admin Dashboard at /admin with platform totals for users, active listings, restricted users and moderated listings.
- Open User Management at /admin/users to search, suspend, ban or restore user accounts with a reason and optional note.
- Open Moderation at /admin/moderation to search every listing and hide, restore or remove any listing with a reason and optional note.
- Read moderation and system notifications from the Notifications page.
- Cannot ban or suspend another admin through this interface.

3. Screen Inventory

The React application ships these screens. Each screen is a page component under code/frontend/src/pages unless noted otherwise.

Screen	Route	Primary Audience
Home	/	Anyone
Auth	/auth	Anyone
Buyer Dashboard	/dashboard	Buyer or Seller
Shop	/shop	Buyer or Seller
Product Detail	/product/:id	Buyer or Seller
Cart Sidebar	Overlay on any buyer page	Buyer or Seller
University Selection	/university	Buyer or Seller
Purchase History	/orders/history	Buyer or Seller
Order Confirmation	/orders/:id/confirmation	Buyer or Seller
Seller Dashboard	/seller	Seller
Listing Form (create and edit)	/seller/listings/new and /seller/listings/:id/edit	Seller
Notifications	/notifications	Buyer, Seller or Admin

Screen	Route	Primary Audience
Admin Dashboard	/admin	Admin
User Management	/admin/users	Admin
Moderation	/admin/moderation	Admin
Not Found	Any unknown route	Anyone

The Messages Inbox and Message Thread screens that appeared in the Mid Submission are not part of the final build. Buyer and seller messaging was intentionally cut for scope and is recorded as a known limitation in Section 4.

4. Use Case List

ID	Name	Primary Actor
UC01	Register Account	Unregistered User
UC02	Log In	Registered User
UC03	Browse Home Page	Buyer or Seller
UC04	Select University	Buyer or Seller
UC05	Browse Listings	Buyer or Seller
UC06	Filter Listings	Buyer or Seller
UC07	View Listing Details	Buyer or Seller
UC08	Save Listing to Wishlist	Buyer
UC09	Message Seller	Buyer
UC10	Add Item to Cart	Buyer
UC11	Checkout and Payment	Buyer
UC12	View Purchase History	Buyer

ID	Name	Primary Actor
UC13	Create Listing	Seller
UC14	Edit Listing	Seller
UC15	Mark Listing as Sold	Seller
UC16	View Seller Dashboard	Seller
UC17	Respond to Messages	Seller
UC18	View Admin Dashboard	Admin
UC19	Remove Listing	Admin
UC20	Ban or Suspend User	Admin

5. Detailed Use Cases

The use cases below are the mid submission use cases updated to reflect the implementation in the final codebase. Flows that referenced Firebase or Google sign in have been rewritten around the local JWT and bcrypt authentication that the final product actually uses. UC09 Message Seller and UC17 Respond to Messages are preserved so the history of the requirement is visible, but both are marked as deferred and explained further in Section 4.

UC01 Register Account

Use Case ID	UC01
Use Case Name	Register Account
Actor	Unregistered User
Description	A new user creates a buyer or seller account through the Auth page at /auth. The page posts to POST /api/auth/register which hashes the password with bcrypt, writes a new User row and returns a JWT session that the browser persists in localStorage. Admin accounts are not registered through this page and are only created through the seeded database.
Pre conditions	The user does not have an active session.

	The Campus Closet Home screen at / and the Auth screen at /auth are reachable.
Post conditions (Success)	A User record exists with a hashed password and a default status of active. The client receives a token and caches it in localStorage under campus_closet_token. The user is redirected to /dashboard for buyers or /seller for sellers.
Post conditions (Failure)	No account is created. The Auth page displays the server error message under the form fields.
Main Flow	1. The user opens /auth. 2. The user switches the form to Register mode. 3. The user enters an email and password and picks a role of buyer or seller. 4. The user optionally selects a default university. 5. The client sends POST /api/auth/register. 6. The server normalizes the email, hashes the password and creates the user. 7. The server returns a 201 response with a signed JWT and the serialized user. 8. The client persists the session and routes the user to their role home.
Alternative Flow	Demo login buttons on the same screen call POST /api/auth/login with one of the seeded demo accounts to shortcut the flow during grading.
Exception Flow	Email already taken: the server returns 409 and the form shows An account with that email already exists. Password under six characters: the server returns 400 and the form shows the length rule. Role other than buyer or seller: the server returns 400 to prevent self promotion to admin.
Business Rules	Email addresses are stored lowercased and must be unique. Passwords are stored only as bcrypt hashes. New accounts always start with a status of active. Only buyer and seller roles can be created through the public register endpoint.

UC02 Log In

Use Case ID	UC02
Use Case Name	Log In
Actor	Registered User (Buyer, Seller or Admin)
Description	An existing user signs in through the Auth page at /auth. The page posts to POST /api/auth/login which verifies the email and bcrypt hashed password, refreshes the account status (so expired suspensions auto heal) and returns a new JWT session.
Pre conditions	The user has an account in the database. The user is not already logged in.
Post conditions (Success)	The client stores a fresh token in localStorage. The user is redirected to the dashboard that matches their role.
Post conditions (Failure)	No session is created and the form displays the server error message in red.
Main Flow	1. The user opens /auth. 2. The user enters email and password. 3. The client posts to /api/auth/login. 4. The server checks that the account exists, refreshes suspension state and compares the password hash. 5. If everything passes and the status is active, the server returns a JWT and the serialized user. 6. The client stores the session and navigates to the role specific home.
Alternative Flow	One click demo buttons call the same login endpoint with the seeded buyer, seller or admin credentials.
Exception Flow	Wrong email or password: the server returns 401 with Invalid email or password. Banned or suspended account: the server returns 403 with the current moderation reason and the suspension end time if one is set.
Business Rules	Credentials are only checked against the local database; there is no external identity provider. Suspended users whose suspensionEndsAt has passed are auto restored to active on the next login attempt.

	All Auth page error copy comes from the backend response message.
--	---

UC03 Browse Home Page

Use Case ID	UC03
Use Case Name	Browse Home Page
Actor	Buyer or Seller
Description	After a successful login, the buyer or seller lands on /dashboard. The page renders a shared DashboardNav, a greeting panel with platform totals, a Your Listings section (for sellers), a Liked Items section populated from the wishlist and a Discover grid of every active listing sorted so that the user university appears first.
Pre conditions	The user is authenticated through a valid JWT stored in localStorage.
Post conditions (Success)	The client loads three endpoints in parallel: /api/listings, /api/wishlist and /api/seller/listings (only for sellers). The three sections render with real persisted data.
Post conditions (Failure)	If any of the requests fail, the corresponding section shows an empty state and the inline error appears above the grid.
Main Flow	1. After login the app navigates to /dashboard. 2. DashboardNav renders with entries for Home, Shop, Cart, Orders, University and Notifications, plus a Sell entry for sellers. 3. The page fetches listings, wishlist and seller listings through apiRequest. 4. The greeting panel shows the user university and four metric cards. 5. Your Listings, Liked Items and Discover render below.
Alternative Flow	If the user has no saved university, the greeting panel swaps the Change University button for Select University that routes to /university. Clicking Cart opens the CartSidebar without leaving the page.
Exception Flow	If a listings or wishlist request fails, the page keeps the navbar and greeting panel and only the impacted section shows an empty state so the user is never stuck on a blank page.

Business Rules	Seller role users see their own listings first so that drafts and hidden listings are reachable directly. The Sign Out button is appended to the navbar automatically by DashboardNav.
-----------------------	--

UC04 Select University

Use Case ID	UC04
Use Case Name	Select University
Actor	Buyer or Seller
Description	The user chooses a default university from /university. The selection is persisted on the User record through PATCH /api/profile and is then used to bias the Discover grid and to preselect the filter chip on the Shop page. The preference persists across sessions because it is stored in the database, not in the browser.
Pre conditions	The user is authenticated. /university is reachable.
Post conditions (Success)	The User row has the new university string written to it. The auth context refreshes from /api/auth/me and the rest of the app immediately reads the new value.
Post conditions (Failure)	The inline error on the panel reports the failure and no change is persisted.
Main Flow	1. The user opens /university. 2. The user searches or scrolls through the chip list. 3. The user taps a chip. 4. The user clicks Save University. 5. The client sends PATCH /api/profile with the new value. 6. The server saves the value and returns the updated user. 7. The client calls refreshUser and navigates to /shop.
Alternative Flow	The user can cancel and return to /dashboard without saving.
Exception Flow	If the request fails the page stays on /university and shows the error copy returned by the server.

Business Rules	The university preference is a profile level setting and is read across the Dashboard and the Shop pages. Users can still browse listings from any school at any time; the preference only changes the default filter.
-----------------------	--

UC05 Browse Listings

Use Case ID	UC05
Use Case Name	Browse Listings
Actor	Buyer or Seller
Description	The Shop page at /shop shows every active listing as a responsive grid of ListingCard components. A filter chip row along the top holds an All chip and one chip per unique school returned by the API. A free text search box filters the grid by title, school, size or condition.
Pre conditions	The user is authenticated. At least one listing is active in the database.
Post conditions (Success)	The grid shows every active listing the API returned. The Cart can be opened from the navbar at any time.
Post conditions (Failure)	If /api/listings fails the grid shows an empty state and the inline error is rendered above it.
Main Flow	1. The user opens /shop. 2. DashboardNav and the filter bar render. 3. The client requests /api/listings. 4. Listings render as ListingCard items. 5. The user clicks a card and is navigated to /product/:id.
Alternative Flow	Picking a school chip narrows the grid. Typing in the search box narrows it further. Clicking the All chip restores the original grid.
Exception Flow	If no listing matches the combined filters, the grid shows the copy No listings matched that filter.
Business Rules	Only listings with status active are returned by /api/listings.

	The user default university auto selects the matching chip on mount when one is set.
--	--

UC06 Filter Listings

Use Case ID	UC06
Use Case Name	Filter Listings
Actor	Buyer or Seller
Description	The Shop page filters the grid in memory based on the selectedSchool chip and the search input. The chip list is computed from the set of unique schools in the listings response so new schools appear automatically when sellers post against them.
Pre conditions	The user is on /shop. At least one listing has been loaded from /api/listings.
Post conditions (Success)	The grid updates immediately because filtering is client side. The selected chip is highlighted with the chip active class.
Post conditions (Failure)	Filtering cannot fail on its own; network failures are handled by UC05.
Main Flow	1. The user clicks a chip. 2. The selectedSchool state updates. 3. The filtered array recomputes. 4. The grid re-renders with only matching items.
Alternative Flow	The user clicks the All chip to clear the school filter. The user types into the search box to filter by any of title, school, size or condition.
Exception Flow	If the filter returns zero matches the grid shows the empty state copy.
Business Rules	The default chip is All when the user has no saved university and is the user university when one is stored. This filter is session local and does not change the profile level preference from UC04.

UC07 View Listing Details

Use Case ID	UC07
Use Case Name	View Listing Details
Actor	Buyer or Seller
Description	The Product Detail page at /product/:id loads a single listing and the current wishlist state in parallel. It displays the photo or a school colored banner, the school tag, the title, the price, a metadata grid of category, size, condition, seller email and posted date and two action buttons: Add to Cart and Like.
Pre conditions	The user is authenticated. The listing id exists and has status active (or the user is the seller or an admin viewing a non active listing).
Post conditions (Success)	The detail layout renders with live data. The Like button reflects the current wishlist state.
Post conditions (Failure)	If the listing cannot be loaded the page shows the error inline and the action buttons are not interactive.
Main Flow	1. The user clicks a card on /dashboard or /shop. 2. The app routes to /product/:id. 3. The client fetches /api/listings/:id and /api/wishlist. 4. The page renders with the photo, school tag, title, price, metadata grid and description. 5. The user can click Add to Cart or Like.
Alternative Flow	The user can tap Back to Shop to return to the grid.
Exception Flow	If the listing is not active and the viewer is not the seller or an admin, /api/listings/:id returns 404 and the page shows that the listing is no longer available.
Business Rules	The server hides non active listings from other users so product detail does not leak draft or hidden inventory. The metadata grid values come directly from the serialized listing payload.

UC08 Save Listing to Wishlist

Use Case ID	UC08
Use Case Name	Save Listing to Wishlist
Actor	Buyer or Seller
Description	The Like button on the Product Detail page toggles a WishlistItem row for the current user and listing. Saved listings appear on the Dashboard Liked Items strip and the detail page shows the toggle state based on the current wishlist contents.
Pre conditions	The user is authenticated. The listing is active. The user is not the seller of the listing.
Post conditions (Success)	A WishlistItem row exists for the user and listing on save. The row is removed on unsave. The Dashboard Liked Items strip reflects the new state on next load.
Post conditions (Failure)	The button shows the error message inline and no change is persisted.
Main Flow	1. The user opens /product/:id. 2. The user clicks Like. 3. The client posts to /api/wishlist. 4. The server calls findOrCreate on WishlistItem so duplicates are impossible. 5. The button switches to Unlike and a small confirmation line renders.
Alternative Flow	If the item is already liked, the button calls DELETE /api/wishlist/:listingId to remove the row.
Exception Flow	Attempting to save your own listing returns 400 with You cannot save your own listing. Attempting to save a non active listing returns 404.
Business Rules	The WishlistItem table has a unique index on (userId, listingId) so the same listing cannot be saved twice. Liked Items are private and only visible to the user who saved them.

UC09 Message Seller

Use Case ID	UC09
Use Case Name	Message Seller
Actor	Buyer
Description	Planned buyer to seller messaging about a specific listing. The use case is retained from the Mid Submission for traceability but the feature was deferred to stay within the scope of the class project. It is listed in Section 4 as a known limitation.
Pre conditions	Not applicable to the shipped build.
Post conditions (Success)	Not applicable to the shipped build.
Post conditions (Failure)	Not applicable to the shipped build.
Main Flow	Not implemented. See Section 4.
Alternative Flow	Not implemented.
Exception Flow	Not implemented.
Business Rules	Buyers and sellers communicate outside the platform in the shipped build. In app messaging would require a Message and Thread model plus UI work, but we did not work on that in this project/build

UC10 Add Item to Cart

Use Case ID	UC10
Use Case Name	Add Item to Cart
Actor	Buyer or Seller
Description	The Add to Cart button on the Product Detail page posts to /api/cart. The server resolves the listing, verifies availability, rejects attempts by the seller to add their own item and either increments the quantity of an existing CartItem or inserts a new one.
Pre conditions	The user is authenticated.

	The listing exists and has status active.
Post conditions (Success)	A CartItem row exists for the user and listing. The Cart Sidebar reflects the change on the next open.
Post conditions (Failure)	The detail page shows the error message inline and the cart is unchanged.
Main Flow	1. The user clicks Add to Cart on /product/:id. 2. The client posts to /api/cart with the listing id and a quantity of one. The endpoint accepts any positive integer quantity; the Product Detail button always sends one. 3. The server validates the listing and the actor. 4. If a CartItem already exists the quantity is incremented; otherwise a new row is created. 5. The Cart Sidebar opens and shows the cart.
Alternative Flow	Adding the same listing twice increments the quantity rather than creating a duplicate row because the CartItem table has a unique index on (userId, listingId).
Exception Flow	Adding your own listing returns 400. Adding a non active listing returns 404.
Business Rules	Quantities are at least one. Buyer and seller roles can both have carts so sellers can demo the full checkout flow during grading.

UC11 Checkout and Payment

Use Case ID	UC11
Use Case Name	Checkout and Payment
Actor	Buyer or Seller
Description	The Cart Sidebar Checkout button posts to /api/orders/checkout. The server runs the whole flow inside a Sequelize transaction so partial writes cannot leak. It reads the cart, verifies every listing is still active, creates an Order, writes one OrderItem per cart line with a price snapshot, marks each listing as sold, clears the cart and sends a notification to each seller.

Pre conditions	The user is authenticated. The cart has at least one item and every item is still active.
Post conditions (Success)	A new Order row is created with a total equal to the sum of quantity times price. One OrderItem row per cart line exists with a titleSnapshot and priceSnapshot captured at purchase time. Every purchased listing has status set to sold. The cart is empty. Each seller receives a new Notification. The client routes to /orders/:id/confirmation.
Post conditions (Failure)	The Sequelize transaction rolls back and nothing is persisted. The sidebar shows the error and the cart is untouched.
Main Flow	1. The user opens the Cart Sidebar. 2. The user reviews line items and the total. 3. The user clicks Checkout. 4. The client posts to /api/orders/checkout. 5. The server starts a transaction and locks the cart rows. 6. The server creates the Order and the OrderItems with price snapshots. 7. The server marks each listing as sold. 8. The server clears the cart and commits. 9. The server writes one notification per distinct seller. 10. The client navigates to the confirmation page for the new order.
Alternative Flow	Cart quantity controls on the sidebar call PATCH /api/cart/:itemId and DELETE /api/cart/:itemId to edit the cart before checkout.
Exception Flow	If the cart is empty the server returns 400 with Your cart is empty. If any listing is no longer active the server rolls back and returns 400 with One or more items in your cart are no longer available.
Business Rules	Price is captured as priceSnapshot on the OrderItem so historical orders stay correct if a future listing price changes. Checkout is simulated end to end in the local database and does not call a real payment processor. This is listed under Section 4.

UC12 View Purchase History

Use Case ID	UC12
Use Case Name	View Purchase History
Actor	Buyer or Seller
Description	The Purchase History page at /orders/history loads the current user orders from GET /api/orders/mine and renders them in a table sorted by most recent. Each row links to the order confirmation view for full line items.
Pre conditions	The user is authenticated.
Post conditions (Success)	Past orders are rendered with id, date, item titles, total and status. Each row offers a View action that routes to /orders/:id/confirmation.
Post conditions (Failure)	The inline error shows under the header and the table stays empty.
Main Flow	1. The user opens /orders/history. 2. The client requests /api/orders/mine. 3. The page renders rows with date, number of items, total and the placed status. 4. The user can click View on any row.
Alternative Flow	When the user has no orders, the page shows a friendly empty state and a Browse Listings button that routes to /shop.
Exception Flow	A network or database error renders the error banner so the user knows why the list is empty.
Business Rules	Orders cannot be edited from the purchase history page. All completed orders are retained in the database.

UC13 Create Listing

Use Case ID	UC13
Use Case Name	Create Listing
Actor	Seller

Description	A seller creates a listing through /seller/listings/new. The ListingForm supports both a Post Listing action that validates every required field and a Save as Draft action that accepts incomplete data. Images are read into a base64 data URL through FileReader and stored on the Listing row.
Pre conditions	The user is authenticated as a seller and has status active.
Post conditions (Success)	A new Listing row is persisted with a sellerId matching the current user and a status of active or draft. The seller is routed back to /seller where the new listing appears at the top of the table.
Post conditions (Failure)	The form shows the server validation message and no row is created.
Main Flow	1. The seller clicks + New Listing on the Seller Dashboard. 2. The seller fills in title, category, school, price, size, condition and description and uploads a photo. 3. The seller clicks Post Listing. 4. The client posts to /api/seller/listings. 5. The server normalizes the payload, validates every required field for active listings and creates the row. 6. The client routes back to /seller.
Alternative Flow	Save as Draft posts the same payload with status draft. The server accepts the row even if fields are missing. Publish from the seller dashboard later calls PATCH /api/seller/listings/:id/status with status set to active, which re runs the validation.
Exception Flow	Active listings missing any required field return 400 with a specific validation message. Prices of zero or below return 400 with Price must be greater than 0. Files over 5 MB or non image types are rejected client side before upload.
Business Rules	Active listings require a title, category, school, size, condition, description and photo. Drafts are stored for the seller only and are never served to buyers through /api/listings. Photos are stored as data URLs inside the listing row for demo simplicity.

UC14 Edit Listing

Use Case ID	UC14
Use Case Name	Edit Listing
Actor	Seller
Description	The same Listing Form page is reused at /seller/listings/:id/edit to edit an existing listing. It loads the current values through /api/seller/listings, lets the seller change any field and posts back through PUT /api/seller/listings/:id.
Pre conditions	The seller is authenticated. The listing exists, is owned by this seller and is not sold or removed.
Post conditions (Success)	The Listing row is updated. Status transitions between draft and active are applied when appropriate.
Post conditions (Failure)	The server message is shown inline and no change is persisted.
Main Flow	1. The seller clicks Edit on a row in the Seller Dashboard. 2. The form loads the listing values. 3. The seller updates fields as needed. 4. The seller clicks Save Changes or Publish Listing. 5. The client sends PUT /api/seller/listings/:id. 6. The server re validates and updates the row.
Alternative Flow	Cancel returns to /seller without saving.
Exception Flow	Attempting to edit a sold or removed listing returns 400 with That listing can no longer be edited. Invalid field values return 400 with the same validation messages as UC13.
Business Rules	Sellers can only edit their own listings. Editing does not reset the created date so feed ordering stays stable.

UC15 Mark Listing as Sold

Use Case ID	UC15
--------------------	------

Use Case Name	Mark Listing as Sold
Actor	Seller
Description	From the Seller Dashboard a seller clicks Mark Sold on an active or hidden listing. The client calls PATCH /api/seller/listings/:id/status with status set to sold. The listing is removed from the public feed and stays in the seller table for records.
Pre conditions	The seller is authenticated. The listing exists, is owned by this seller and is not already sold or removed.
Post conditions (Success)	Listing status is set to sold and the items sold count updates on the dashboard.
Post conditions (Failure)	The inline error is shown and no change is applied.
Main Flow	1. The seller clicks Mark Sold on a row. 2. The client issues the status PATCH. 3. The server updates the listing. 4. The row refreshes in place in the table.
Alternative Flow	Sellers can use the same control to Hide or Restore a listing by choosing hidden or active rather than sold.
Exception Flow	Attempting to transition a sold or removed listing returns 400 with Sold listings cannot change status.
Business Rules	Sold listings are never deleted; they remain for seller records. A listing that was auto checked out during UC11 also lands in the sold state through the same transition.

UC16 View Seller Dashboard

Use Case ID	UC16
Use Case Name	View Seller Dashboard
Actor	Seller

Description	The Seller Dashboard at /seller loads the seller listings through /api/seller/listings and the sold line items through /api/seller/orders in parallel. It renders four stat cards (Total Earned, Active Listings, Items Sold, Draft Listings), a table of listings with inline Edit, Publish, Hide, Restore and Mark Sold actions and a table of recent sold line items.
Pre conditions	The user is authenticated as a seller and has status active.
Post conditions (Success)	Both tables render with live data. Stat cards reflect the current counts.
Post conditions (Failure)	The inline error renders above the tables.
Main Flow	1. The seller clicks Dashboard in the navbar. 2. The page issues the two requests in parallel. 3. The tables and stat cards render. 4. The seller can launch any of the action buttons inline.
Alternative Flow	Clicking + New Listing routes to UC13. Clicking Edit on a draft or hidden row routes to UC14.
Exception Flow	If either request fails the matching table shows an empty state.
Business Rules	Only listings owned by the seller are shown. Draft listings are never returned by the public /api/listings endpoint.

UC17 Respond to Messages

Use Case ID	UC17
Use Case Name	Respond to Messages
Actor	Seller
Description	Planned seller messaging response flow. Deferred in the final build for the same reason as UC09 and listed under Section 4 as a known limitation.
Pre conditions	Not applicable to the shipped build.
Post conditions (Success)	Not applicable to the shipped build.

Post conditions (Failure)	Not applicable to the shipped build.
Main Flow	Not implemented. See Section 4.
Alternative Flow	Not implemented.
Exception Flow	Not implemented.
Business Rules	Out of band communication is expected between the parties in the current build.

UC18 View Admin Dashboard

Use Case ID	UC18
Use Case Name	View Admin Dashboard
Actor	Admin
Description	The Admin Dashboard at /admin loads every user through /api/admin/users and every listing through /api/admin/listings in parallel. It shows four stat cards (Total Users, Active Listings, Restricted Users and Flagged Items), a recent users table and a moderated listings table.
Pre conditions	The user has the admin role.
Post conditions (Success)	Both tables render and every stat card shows the current platform totals.
Post conditions (Failure)	The inline error is shown above the tables.
Main Flow	1. The admin logs in and lands at /admin. 2. The page issues both admin requests. 3. The two tables and four stat cards render. 4. The admin can jump to User Management or Moderation through the navbar or the View All buttons on each table.
Alternative Flow	Clicking User Management or Moderation in the navbar routes directly to those pages without passing through the dashboard tables.

Exception Flow	If either request fails the associated table shows an empty state and the inline error renders above the tables.
Business Rules	Only admin users can reach /admin or any /api/admin route. Restricted Users counts include both suspended and banned accounts because both are enforced through the same middleware block.

UC19 Remove Listing

Use Case ID	UC19
Use Case Name	Remove Listing
Actor	Admin
Description	From /admin/moderation an admin removes a listing that violates community rules. The client prompts for a reason and an optional note, then sends PATCH /api/admin/listings/:id/status with status removed and the text. The server sets the new status, stores the reason and note on the listing row and writes a notification to the seller.
Pre conditions	The user has the admin role. The listing exists and is not sold.
Post conditions (Success)	Listing status is removed. adminReason and adminNote are persisted. The seller has a new Notification explaining the removal.
Post conditions (Failure)	The inline error is shown and the listing state is unchanged.
Main Flow	1. The admin opens /admin/moderation. 2. The admin types a search or scrolls to a listing. 3. The admin clicks Remove. 4. The browser prompts for a reason and optional note. 5. The client sends the status PATCH with status removed. 6. The server updates the listing and writes the notification.
Alternative Flow	The admin can Hide a listing instead with the same table, which sets status to hidden without touching the reason and note fields.

	Hiding or restoring is a lighter action that the seller can usually reverse.
Exception Flow	A remove call without a reason returns 400. Attempting to moderate a sold listing returns 400. Attempting to restore a listing that was auto hidden because of seller account restriction returns 400 with an instruction to restore the account first.
Business Rules	Removed listings are never deleted; they are archived with their reason and note so that audit history survives. Restoring a removed listing to active clears the reason and note fields and sends a restore notification to the seller. Restoring a listing that was only hidden (without a prior admin reason or note) flips the status to active without sending an additional notification.

UC20 Ban or Suspend User

Use Case ID	UC20
Use Case Name	Ban or Suspend User
Actor	Admin
Description	From /admin/users an admin suspends or bans an account. The client prompts for a reason, a duration (1, 7 or 30 days for suspensions) and an optional note. The server updates the User row, hides the user active listings automatically so buyers cannot reach them, writes a notification to the user and blocks future authenticated requests through the requireAuth middleware until the account is restored.
Pre conditions	The user has the admin role. The target user is not an admin. The admin is not operating on themselves.
Post conditions (Success)	User status is suspended or banned. statusReason, statusNote and, for suspensions, suspensionEndsAt are persisted. Every active listing by this user moves to hidden with hiddenByAccountStatus set to true. The user has a new Notification.

Post conditions (Failure)	The inline error is shown and no changes are persisted.
Main Flow	1. The admin opens /admin/users. 2. The admin searches for the target. 3. The admin clicks Suspend or Ban. 4. Browser prompts collect the reason, the duration for suspensions and an optional note. 5. The client sends PATCH /api/admin/users/:id/status. 6. The server updates the user and hides their active listings. 7. The server writes a notification and returns the updated user.
Alternative Flow	Clicking Restore on a suspended or banned user opens a browser prompt for an optional note, then posts status active. The server restores listings that were hidden specifically because of the account restriction (hiddenByAccountStatus true) and writes an Account restored notification.
Exception Flow	Suspensions without a valid duration or reason return 400. Bans without a reason return 400. Attempting to ban or suspend an admin returns 400.
Business Rules	Suspended accounts auto recover when the current time passes suspensionEndsAt; the requireAuth middleware performs this check on every request. Listings auto hidden by account restriction are restored only when the account itself is restored. Admins cannot manually promote these listings to active through the moderation page while the owning account is still suspended or banned; the moderation endpoint explicitly rejects that transition.

6. Technology Stack and Architecture

Languages and Runtime

- JavaScript only across the whole project.
- Node.js 20 or newer runs the backend.
- Modern browsers run the React frontend after a Vite build.

Frontend

- React 19 with functional components and hooks.

- Vite for local development and production bundling.
- React Router 7 for client side routing.
- Bootstrap plus custom CSS for layout and visual styling.
- A tiny fetch wrapper at `code/frontend/src/lib/api.js` that injects the JWT and normalizes error responses.
- An `AuthContext` that persists the JWT in `localStorage` and exposes `login`, `register`, `logout` and `refreshUser` helpers.

Backend

- Node.js plus Express 5 for HTTP routing.
- Sequelize 6 as the ORM.
- SQLite as the single file database stored at `code/database/campus_closet.sqlite`.
- JSON Web Tokens signed with `jsonwebtoken`.
- `bcryptjs` for password hashing.
- A tiny `requireAuth` and `requireRole` pair of middleware functions for authorization.

Project Layout

- Root workspace with shared scripts (`dev`, `build`, `seed`, `start`).
- `code/frontend` contains the Vite React app.
- `code/backend` contains the Express API, Sequelize models, route handlers, middleware and a seed script.
- `code/database` holds the SQLite artifact that ships with the submission.

How the Pieces Fit Together

The React app talks to the backend through `/api` over HTTP. In development Vite proxies `/api` to `http://localhost:3001` so the developer only needs one URL open. In the production build served by a static host the same proxy would be provided by the hosting layer. Every API call travels through `apiRequest`, which reads the token out of the `AuthContext`, adds the `Authorization` header and surfaces errors as an `ApiError` subclass so pages can render them inline.

On the server side every Express route file mounts under `/api` with a prefix like `/auth`, `/listings` or `/admin`. Most routes are protected by `requireAuth` and sometimes by `requireRole`. The middleware loads the User record by id, refreshes any expired suspension back to active and blocks requests whose status is anything other than active. That is the single enforcement point that keeps banned or suspended users out of the API regardless of which route they hit.

Sequelize provides the Model layer. Associations between `User`, `Listing`, `CartItem`, `WishlistItem`, `Order`, `OrderItem` and `Notification` are declared once in `code/backend/src/models/index.js`. Every route reads

and writes through those models and returns serialized responses built by the helpers in `code/backend/src/utils/serializers.js` so the API payloads never leak implementation detail like password hashes.

7. Section 1: MVC Implementation Walkthrough

Each use case below documents the route that receives the request, the controller logic that runs, the models that are read from and written to and the views that the user sees. No code snippets are included; the goal is to describe what the components do and why.

7.1 Use Case UC02 Log In

The Route

The Auth page posts to `POST /api/auth/login`. The route is declared in `code/backend/src/routes/auth.js` and is not guarded by `requireAuth` since the user is still anonymous. The handler accepts an email and a password in the JSON body.

The Controller

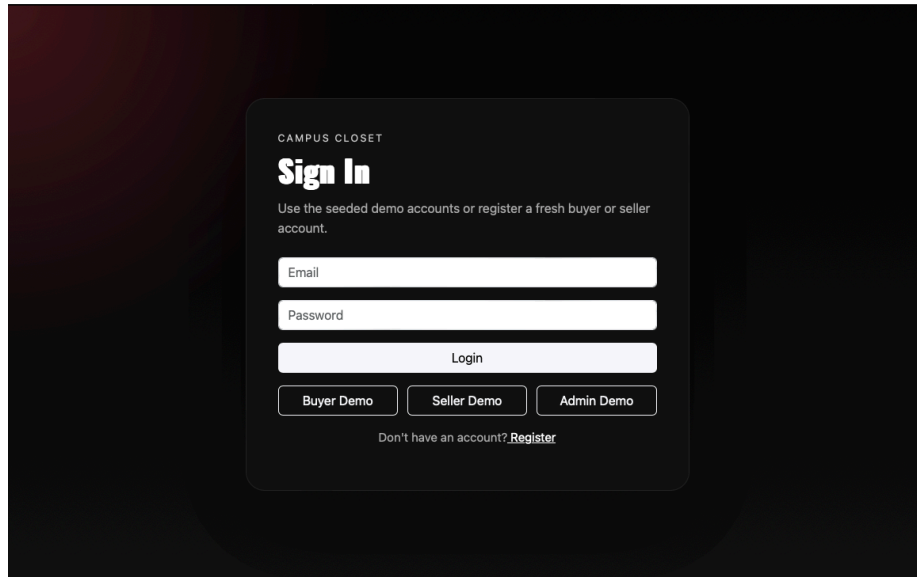
- Normalize the email to lowercase and trim whitespace.
- Look up the User row by email.
- Call `refreshUserStatus` so that any suspension whose end date has passed is auto restored before credentials are even checked.
- Compare the provided password against the stored bcrypt hash.
- Reject the request with a 403 and the human readable status message if the account is suspended or banned.
- Otherwise sign a JWT with the user id, wrap it with the serialized user and return a 200 response.

The Models

Model	Operation	What Happens
User	Read and possibly update	Loaded by email. If a suspension has expired, <code>refreshUserStatus</code> sets status back to active and clears the related fields in a single save call.

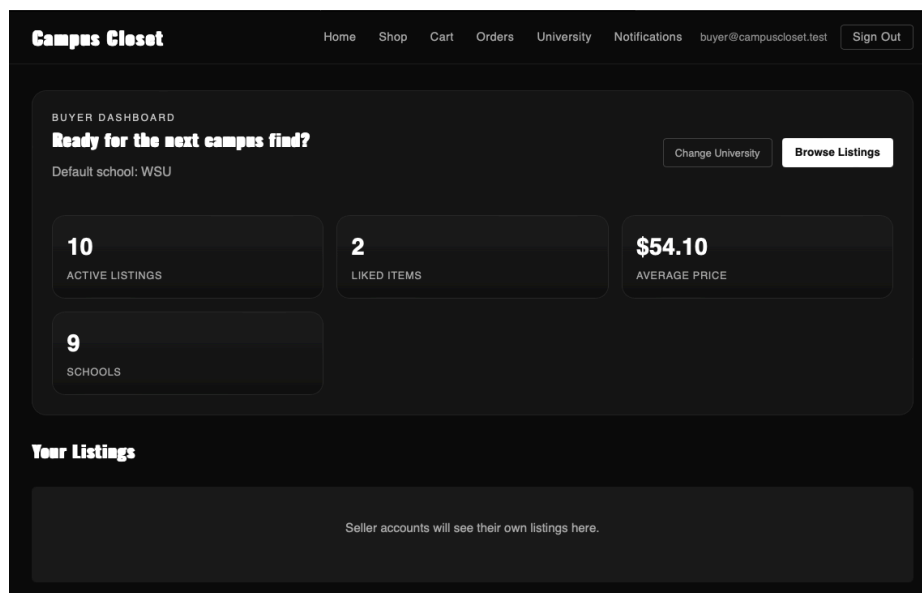
The Views

View one is the Auth page at /auth. It renders the form and shows any server message as red copy under the title. View two is the role specific home page (/dashboard, /seller or /admin) that the AuthContext redirects to once the session lands in localStorage.



The screenshot shows the 'Sign In' page for 'CAMPUS CLOSET'. The page has a dark background with a central white form. The form includes fields for 'Email' and 'Password', a 'Login' button, and three buttons for 'Buyer Demo', 'Seller Demo', and 'Admin Demo'. Below the buttons is a link to 'Register' for users who don't have an account. The text 'Use the seeded demo accounts or register a fresh buyer or seller account.' is displayed above the form fields.

Figure 1: /auth page (login form)



The screenshot shows the 'BUYER DASHBOARD' for 'Campus Closet'. The dashboard has a dark background with a white header. The header includes the 'Campus Closet' logo, navigation links (Home, Shop, Cart, Orders, University, Notifications), the user email 'buyer@campuscloset.test', and a 'Sign Out' button. The main content area features a 'Ready for the next campus find?' section with a 'Default school: WSU' and buttons for 'Change University' and 'Browse Listings'. Below this are four statistics: '10 ACTIVE LISTINGS', '2 LIKED ITEMS', '\$54.10 AVERAGE PRICE', and '9 SCHOOLS'. The 'Your Listings' section is empty, with a message stating 'Seller accounts will see their own listings here.'

Figure 2: /dashboard after successful login

7.2 Use Case UC04 Select University

The Route

PATCH /api/profile declared in code/backend/src/routes/profile.js. Guarded by requireAuth. No role restriction because every authenticated user is allowed to update their own university preference.

The Controller

- Read the new university value off the request body, trimming whitespace.
- Treat an empty string as null so the column does not store blanks.
- Update the current user university field and save the row.
- Return the serialized user so the client can refresh its auth context without an extra round trip.

The Models

Model	Operation	What Happens
User	Update	Only the university column is touched. statusReason and other moderation fields are untouched.

The Views

View one is /university which shows the searchable chip list and the Save button. View two is /shop; the page reads the refreshed user.university and preselects the matching chip so the first render already reflects the new preference.

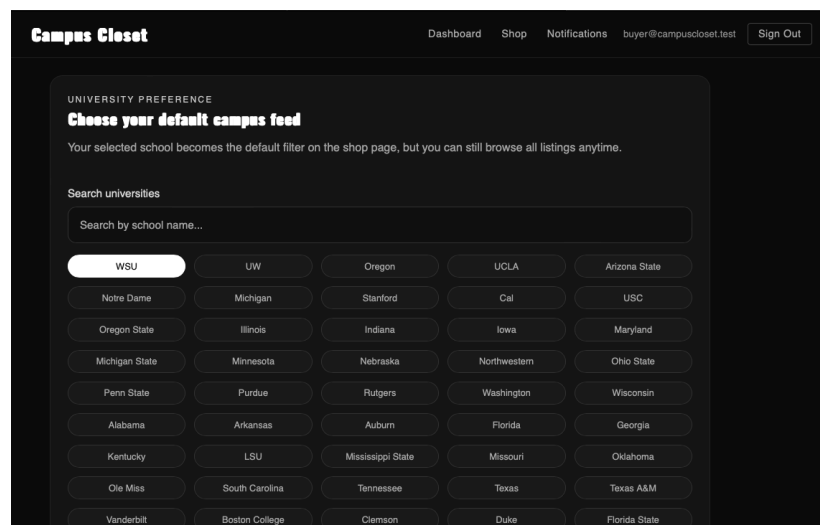


Figure 3: University Selection Page

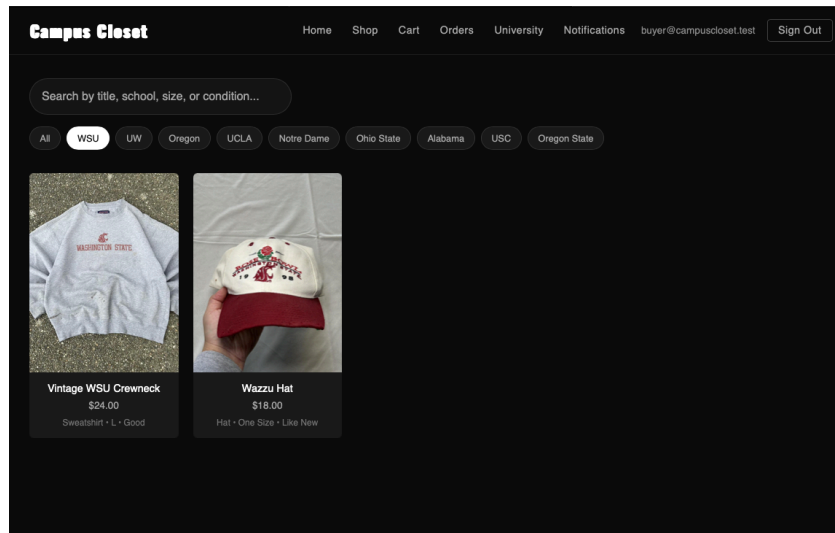


Figure 4: Shop Page with University filter chip preselected

7.3 Use Case UC08 Save Listing to Wishlist

The Route

POST /api/wishlist to save a listing and DELETE /api/wishlist/:listingId to remove one. Both routes live in code/backend/src/routes/wishlist.js and are guarded by requireAuth plus requireRole('buyer', 'seller').

The Controller

- Read the listing id and reject the request if it is missing or falsy.
- Load the listing by id and reject with 404 if it is not active.
- Reject with 400 if the current user is the seller of the listing.
- Call WishlistItem.findOrCreate keyed on (userId, listingId) so duplicate saves are impossible.
- Reload the wishlist and return the serialized listings so the UI can repaint the Liked Items strip immediately.

The Models

Model	Operation	What Happens
Listing	Read	Loaded to verify it exists and is currently active.
WishlistItem	Create or Destroy	findOrCreate on save and destroy on unsave. A unique index on (userId, listingId) enforces the one save per listing rule at the database level.

The Views

View one is the Product Detail page at /product/:id which holds the Like button and the toggle state.

View two is the Dashboard at /dashboard where the Liked Items horizontal strip picks up the new row on the next page load.

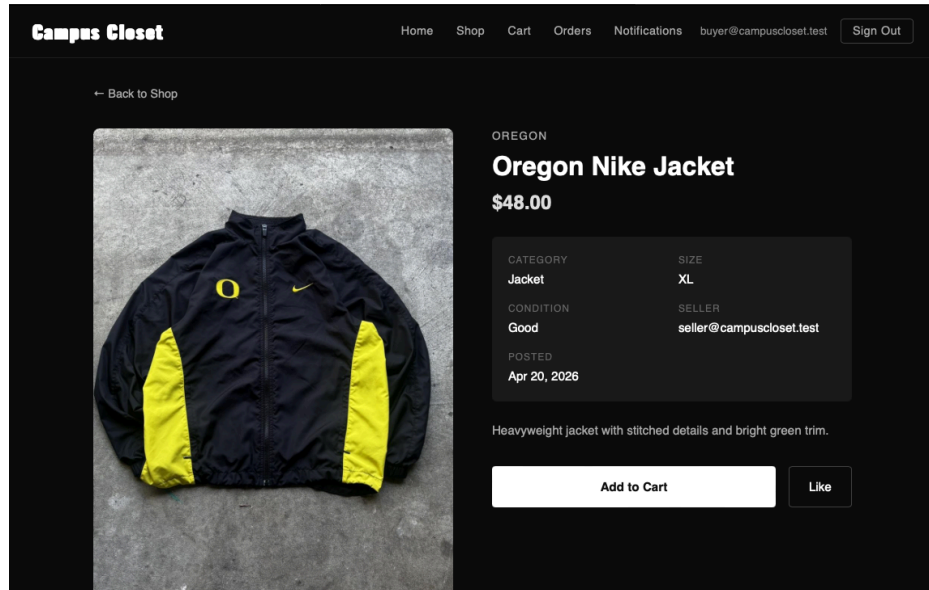


Figure 5: Product details page showing 'add to cart' and 'like' button

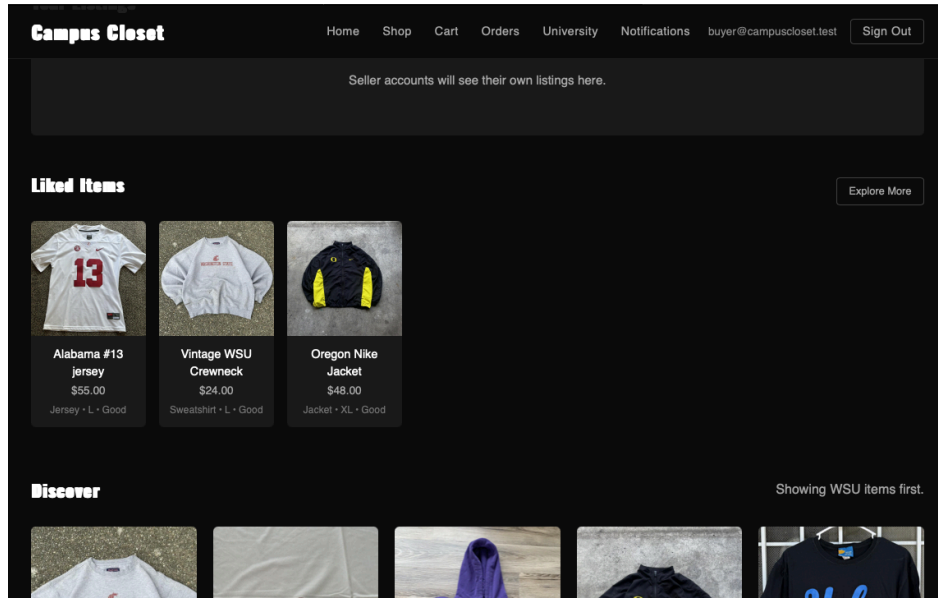


Figure 6: Dashboard Liked Items strip showing 'liked' listings

7.4 Use Case UC11 Checkout and Payment

The Route

POST /api/orders/checkout declared in code/backend/src/routes/orders.js. Guarded by requireAuth and requireRole('buyer', 'seller'). Accepts nobody; the cart is derived from the authenticated user.

The Controller

- Open a Sequelize managed transaction.
- Load every CartItem for the user with the related Listing, locking the rows for update so concurrent checkouts cannot double spend a cart.
- Return 400 with Your cart is empty if the cart has no items.
- Return 400 with One or more items in your cart are no longer available if any related listing has status other than active.
- Compute the total as the sum of listing price times quantity across every cart line.
- Create the Order row with the buyer id, total and a status of placed.
- Bulk creates OrderItem rows capturing sellerId, quantity, titleSnapshot and priceSnapshot per line.
- Update every purchased Listing row to status sold in a single update statement.
- Delete the cart for the buyer.
- Commit the transaction.
- After committing, rehydrate the order with its items and sellers and write one Notification per seller about the sale.
- Return the serialized order with a 201 so the client can route to the confirmation page.

The Models

Model	Operation	What Happens
CartItem	Read then Delete	All cart items for the user are read under a row level lock and then destroyed after the order commits.
Listing	Read then Update	Each cart listing is verified as active and then promoted to sold in a single update.
Order	Create	New Order row with the total and placed status.
OrderItem	Create	One per cart line, storing a title and price snapshot so history stays correct if the seller changes the listing later.

Model	Operation	What Happens
Notification	Create	One notification per seller is written after commit so the transaction does not depend on the notification insert.

The Views

View one is the Cart Sidebar component which renders the cart, the running total and the Checkout button. View two is the Order Confirmation page at `/orders/:id/confirmation` which loads the new order through `/api/orders/:id` and shows the buyer the receipt.

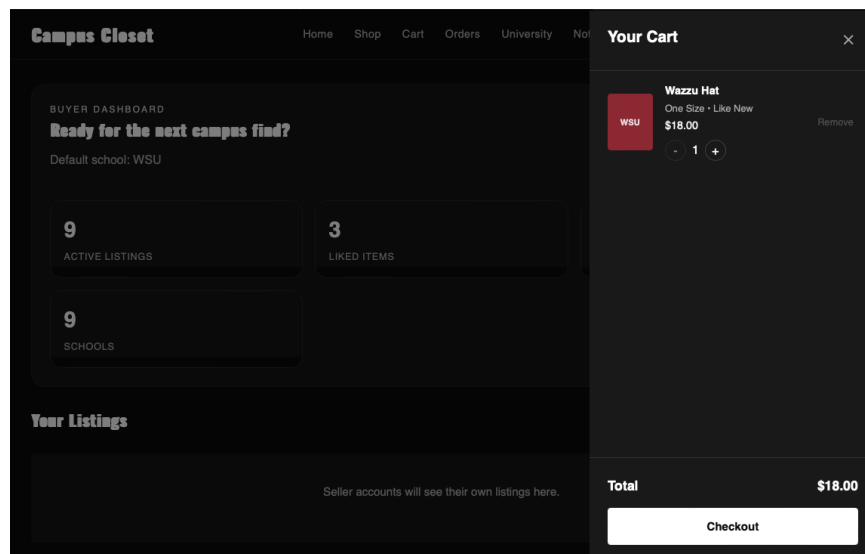


Figure 7: Cart Sidebar showing items, checkout, and total amount of money.

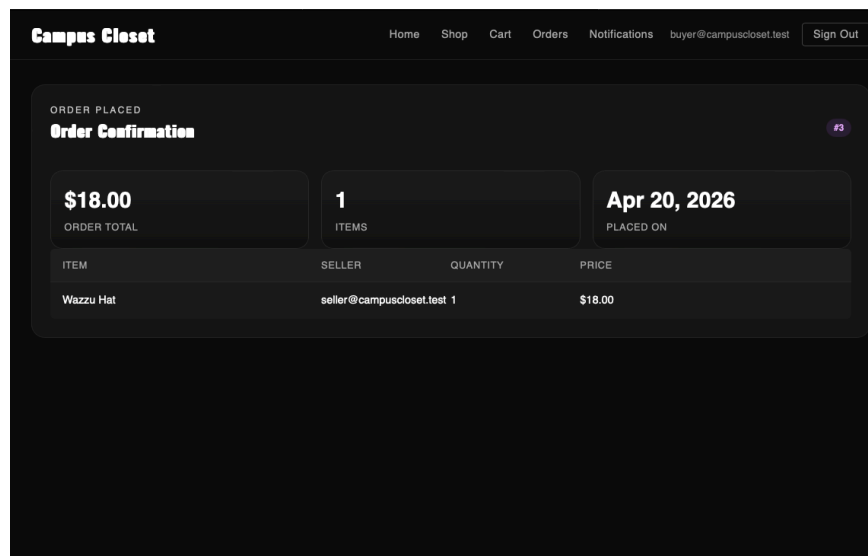


Figure 8: Order Confirmation page

7.5 Use Case UC13 Create Listing

The Route

POST /api/seller/listings declared in code/backend/src/routes/seller.js. Guarded by requireAuth and requireRole('seller'). Accepts title, category, school, price, size, condition, description, imageUrl and status in the JSON body.

The Controller

- Normalize the payload by trimming strings and turning empty values into null.
- Coerce status to either active or draft because those are the only two values the create endpoint allows.
- Validate that active listings have every required field plus a positive price; drafts skip validation so they can be saved partially filled.
- Create the Listing row with sellerId set to the authenticated user.
- Reload the new row with its seller and return it.

The Models

Model	Operation	What Happens
Listing	Create	A new row is inserted with status active or draft and sellerId equal to the authenticated user.
User	Read	Eager loaded as seller so the serializer can include seller metadata on the response.

The Views

View one is the Listing Form at /seller/listings/new. It drives the upload (reading the chosen image into a data URL through FileReader) and the Save as Draft or Post Listing buttons. View two is the Seller Dashboard at /seller which renders the new listing in the listings table on return.

Campus Clesot

DashboardShopNotificationsNew Listingseller@campuscloset.testSign Out

Create Listing

READY TO PUBLISH

Active listings require every field plus a photo. Drafts can be saved with partial information.

Title

Demo Listing

Category

Pullover

School

WSU

Price

9000

Size

m

Condition

New

Upload Photo

Choose Filemaxresdefault-2.jpg

Testing

Photo ready

This image will be stored with the listing for the local demo build.

Description

brand new demo listing

Figure 9: Listing form filled out

Campus Clesot

DashboardShopNotificationsNew Listingseller@campuscloset.testSign Out

Seller Dashboard

\$86.00

TOTAL EARNED

9

ACTIVE LISTINGS

3

ITEMS SOLD

1

DRAFT LISTINGS

Your Listings

+ New Listing

LISTING	STATUS	PRICE	CREATED	ACTIONS
Testing Demo Listing Pullover • m • New	active	\$9,000.00	Apr 20, 2026	Edit Hide Mark Sold
Vintage WSU Crewneck Sweatshirt • L • Good	sold	\$24.00	Apr 20, 2026	
UW Game Day Hoodie Hoodie • M • Excellent	active	\$32.00	Apr 20, 2026	Edit Hide Mark Sold
Oregon Nike Jacket Jacket • XL • Good	active	\$48.00	Apr 20, 2026	Edit Hide Mark Sold

Figure 10: Seller dashboard showing newly created listing (“Demo Listing”)

7.6 Use Case UC20 Ban or Suspend User

The Route

PATCH /api/admin/users/:id/status declared in code/backend/src/routes/admin.js. Guarded by requireAuth and requireRole('admin'). Accepts status, reason, note and durationDays in the JSON body.

The Controller

- Validate that status is one of active, suspended or banned.
- Load the target user and reject if the status is not active and the target user is an admin (admins cannot ban or suspend each other through this endpoint).
- For suspended, require a reason plus a duration of 1, 7 or 30 days. For banned, require a reason.
- Update the User row with the new status, reason, note and suspensionEndsAt (only for suspended).
- If the new status is restrictive, call hideListingsForAccount which sets every active listing for the user to hidden with hiddenByAccountStatus true.
- If the new status is active, call restoreListingsForAccount which restores only the listings that were hidden specifically because of the account restriction.
- Write a Notification for the affected user explaining the action.
- Return the updated serialized user to the admin UI.

The Models

Model	Operation	What Happens
User	Update	Status, reason, note and suspension end date are updated on a single save call.
Listing	Update	Seller listings are bulk updated to hidden on restriction or back to active on restoration.
Notification	Create	The affected user gets a new notification so they can see the reason in the app.

The Views

View one is the User Management table at /admin/users. It renders the search input, the table rows and the inline Suspend, Ban and Restore actions. View two is the Notifications page at /notifications for the affected user, which shows the moderation notice on their next sign in.

Campus Closet — Admin

[Dashboard](#) [Moderation](#) [admin@campuscloset.test](#) [Sign Out](#)

User Management

All Users

EMAIL ACTIONS	ROLE	JOINED	STATUS	MODERATION
testing@ban.me WSU Restore	buyer	Apr 20, 2026	banned	Testing, Testing.
buyer@campuscloset.test WSU Suspend Ban	buyer	Apr 20, 2026	active	Good standing
seller@campuscloset.test UW Suspend Ban	seller	Apr 20, 2026	active	Good standing
admin@campuscloset.test Suspend Ban	admin	Apr 20, 2026	active	Good standing

Figure 11: User Management page with suspend/ban buttons

Campus Closet

[Dashboard](#) [Shop](#) [Orders](#) [University](#) [testing@ban.me](#) [Sign Out](#)

ACTIVITY

Notifications

[Mark All Read](#)

This account has been banned.

You do not have any notifications yet.

Figure 12: Notifications page for banned user showing restriction notice.

8. Section 2: Route and API Table

Every route that the frontend or an external tool can hit is listed below. Frontend routes are declared in `code/frontend/src/App.jsx`. Backend routes are declared under `code/backend/src/routes`. The auth column names the role required; Session means any authenticated user is allowed.

8.1 Frontend Routes

Path	Page	Auth
/	Home	None
/auth	Auth (login and register)	None
/dashboard	Buyer Dashboard (sellers see Your Listings; admins are redirected to /admin)	Session
/shop	Shop	Buyer or Seller
/product/:id	Product Detail	Buyer or Seller
/orders/history	Purchase History	Buyer or Seller
/orders/:id/confirmation	Order Confirmation	Buyer or Seller
/university	University Selection	Buyer or Seller
/notifications	Notifications	Session
/seller	Seller Dashboard	Seller
/seller/listings/new	Create Listing (Listing Form)	Seller
/seller/listings/:id/edit	Edit Listing (Listing Form)	Seller
/admin	Admin Dashboard	Admin
/admin/users	User Management	Admin
/admin/moderation	Listing Moderation	Admin
*	Not Found	None

8.2 Backend API Endpoints

Authentication

Method	Route	Description	Auth
POST	/api/auth/register	Create a buyer or seller account and return a JWT	None
POST	/api/auth/login	Verify credentials, auto heal expired suspensions and return a JWT	None
GET	/api/auth/me	Return the current user; used by the AuthContext on boot	Session
GET	/api/health	Liveness probe returning { status: 'ok' }	None

Profile

Method	Route	Description	Auth
PATCH	/api/profile	Update the signed in user university preference	Session

Listings (public catalog)

Method	Route	Description	Auth
GET	/api/listings	Return every active listing. Accepts an optional school query parameter	Session
GET	/api/listings/:id	Return a single listing. Non active listings are only visible to the seller or an admin	Session

Wishlist

Method	Route	Description	Auth
GET	/api/wishlist	Return every saved listing for the current user	Buyer or Seller
POST	/api/wishlist	Save a listing; upsert through findOrCreate	Buyer or Seller

Method	Route	Description	Auth
DELETE	/api/wishlist/:listingId	Remove the saved listing for this user and id pair	Buyer or Seller

Cart

Method	Route	Description	Auth
GET	/api/cart	Load every cart line with the associated listing and seller	Buyer or Seller
POST	/api/cart	Add a listing to the cart or increment quantity	Buyer or Seller
PATCH	/api/cart/:itemId	Update the quantity for a specific cart line	Buyer or Seller
DELETE	/api/cart/:itemId	Remove a cart line	Buyer or Seller

Orders

Method	Route	Description	Auth
GET	/api/orders/mine	List orders for the signed in buyer or seller	Buyer or Seller
POST	/api/orders/checkout	Atomic checkout under a Sequelize transaction	Buyer or Seller
GET	/api/orders/:id	Return the order detail for the buyer, an admin or a seller whose items are on the order	Session

Notifications

Method	Route	Description	Auth
GET	/api/notifications	List notifications for the current user, most recent first	Session

Method	Route	Description	Auth
PATCH	/api/notifications/:id/read	Mark a single notification as read	Session
PATCH	/api/notifications/read-all	Mark every notification for the user as read	Session

Seller Listings

Method	Route	Description	Auth
GET	/api/seller/listings	List every listing belonging to the signed in seller (active, hidden, draft, sold or removed)	Seller
POST	/api/seller/listings	Create a new listing as active or draft	Seller
PUT	/api/seller/listings/:id	Update an existing listing unless it is sold or removed	Seller
PATCH	/api/seller/listings/:id/status	Transition a listing to active, hidden or sold	Seller
GET	/api/seller/orders	Return every order line belonging to this seller with the buyer and listing eagerly loaded	Seller

Admin User and Listing Moderation

Method	Route	Description	Auth
GET	/api/admin/users	List every user in the system	Admin
PATCH	/api/admin/users/:id/status	Suspend, ban or restore a user and auto hide or restore their listings	Admin
GET	/api/admin/listings	List every listing regardless of status	Admin
PATCH	/api/admin/listings/:id/status	Hide, remove or restore a listing with a reason and optional note	Admin

9. Section 3: Data Model and Relationships

Sequelize defines seven persistent models. The following subsections describe each model, its fields and the relationships it participates in.

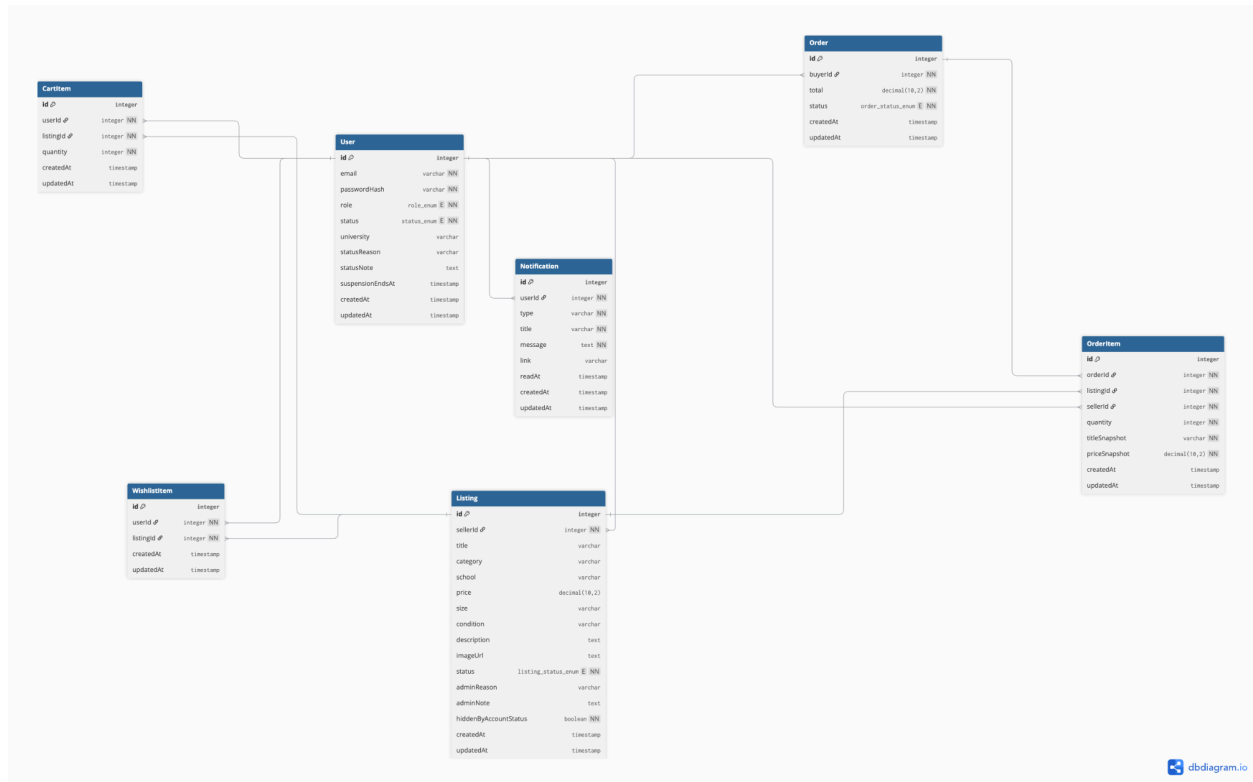


Figure 13: ER Diagram

9.1 Models and Fields

User

Field	Type	Notes
id	INTEGER	Primary key, auto increment
email	STRING	Unique, validated as an email and always stored lowercased
passwordHash	STRING	bcrypt hash; the plain password is never persisted
role	ENUM	buyer, seller or admin
status	ENUM	active, suspended or banned, default active

Field	Type	Notes
university	STRING	Nullable; default campus filter for the user
statusReason	STRING	Reason recorded when the status is not active
statusNote	TEXT	Optional admin note stored alongside the reason
suspensionEndsAt	DATE	Suspension auto heals when the current time passes this timestamp
createdAt / updatedAt	DATE	Managed by Sequelize

User is the central identity record. The role column controls which pages and endpoints the account can reach; the requireAuth middleware plus refreshUserStatus guarantees that suspensions and bans take effect immediately on the next request.

Listing

Field	Type	Notes
id	INTEGER	Primary key, auto increment
sellerId	INTEGER	Foreign key to User.id
title	STRING	Required for active listings; drafts may have it null
category	STRING	Category tag such as Hoodie or Hat
school	STRING	The campus the listing is tied to
price	DECIMAL(10,2)	Nullable for drafts; must be positive for active listings
size	STRING	Size tag chosen by the seller
condition	STRING	Human readable condition such as Good or Excellent
description	TEXT	Long form copy
imageUrl	TEXT	Data URL of the uploaded image
status	ENUM	draft, active, hidden, sold or removed, default active

Field	Type	Notes
adminReason	STRING	Set when an admin removes or hides the listing
adminNote	TEXT	Optional admin note
hiddenByAccountStatus	BOOLEAN	True when the listing was auto hidden because the seller was suspended or banned

Listing is the product record. The status enum combined with hiddenByAccountStatus is what lets moderation, seller visibility and auto restore on account reinstatement all coexist without stepping on each other.

CartItem

Field	Type	Notes
id	INTEGER	Primary key
userId	INTEGER	Foreign key to User.id
listingId	INTEGER	Foreign key to Listing.id
quantity	INTEGER	At least one; bumped on repeat add instead of inserting a duplicate

CartItem has a unique index on (userId, listingId) so each user can hold at most one row per listing. Quantity is bumped on repeat adds by the cart controller.

WishlistItem

Field	Type	Notes
id	INTEGER	Primary key
userId	INTEGER	Foreign key to User.id
listingId	INTEGER	Foreign key to Listing.id

WishlistItem is a simple join table with a unique index on (userId, listingId). It drives the Liked Items strip on /dashboard.

Order

Field	Type	Notes
id	INTEGER	Primary key
buyerId	INTEGER	Foreign key to User.id
total	DECIMAL(10,2)	Sum of quantity times price across the order lines at the time of purchase
status	ENUM	placed; the build ships with a single terminal status for simplicity

OrderItem

Field	Type	Notes
id	INTEGER	Primary key
orderId	INTEGER	Foreign key to Order.id
listingId	INTEGER	Foreign key to Listing.id
sellerId	INTEGER	Foreign key to User.id; denormalized so seller queries are cheap
quantity	INTEGER	Number of units purchased
titleSnapshot	STRING	Title of the listing at purchase time
priceSnapshot	DECIMAL(10,2)	Unit price at purchase time; the history does not drift if the seller later edits the listing

OrderItem is the line item for each purchase. Snapshot fields make the purchase history and the seller order list deterministic even if a future edit would have changed the source listing.

Notification

Field	Type	Notes
id	INTEGER	Primary key
userId	INTEGER	Foreign key to User.id

Field	Type	Notes
type	STRING	Free form type such as order, listing, wishlist, account or system
title	STRING	Headline copy
message	TEXT	Body copy
link	STRING	Optional in app link the Open button can route to
readAt	DATE	Null until the user marks the notification as read

9.2 Relationships

Relationship	Type	Description
User has many Listings as seller	1 to N	A seller User can own many Listings through sellerId.
User has many CartItems	1 to N	Every authenticated user can own a cart.
Listing has many CartItems	1 to N	Conceptually one listing can sit in many carts at once, though the checkout transaction removes that overlap once someone buys.
User has many WishlistItems	1 to N	Each user has their own wishlist.
Listing has many WishlistItems	1 to N	A listing can be saved by many users.
User has many Orders as buyer	1 to N	Buyers accumulate Orders over time.
Order has many OrderItems	1 to N	Standard order to line item relationship.
Listing has many OrderItems	1 to N	A listing can appear in many historical order lines, even if only one purchase actually occurred for it in practice.
User has many OrderItems as seller	1 to N	Denormalized so the seller dashboard list is a single query rather than a join through Order.
User has many Notifications	1 to N	Every notification is scoped to a single recipient.

All associations are declared in `code/backend/src/models/index.js` immediately after the model definitions. The seed script at `code/backend/src/scripts/seed.js` exercises every relationship so a fresh database already contains realistic data for every screen of the app.

10. Section 4: Known Issues and Limitations

- Buyer and seller messaging (UC09 and UC17) is intentionally cut from the final build. Communication happens out of band for now.
- Notifications are stored in the application only. No real email is sent. (A real product Could use Resend)
- Photo upload is stored as a base64 data URL on the listing record so the entire demo runs without a cloud image service (real application would use R2 storage from Cloudflare)
- Listing removal and user restore still use lightweight browser prompts (`window.prompt`) to collect the admin reason and note. Suspend and ban use an inline form panel on the User Management page with a required reason field, an optional note and, for suspensions, a 1/7/30 day selector. A production version would replace every remaining prompt with a modal driven workflow.
- Checkout is simulated end to end inside the local database. No real payment processor is contacted. (Real product could use Stripe, or other payment processor)
- The Admin UI prevents an admin from ban or suspend actions on themselves by disabling the buttons. The backend rule blocks moderation of admin accounts in general.

11. Running the Project Locally

Install dependencies from the repo root with `npm install`. Copy the env template with `cp .env.example .env`. Reset the demo database and seed accounts with `npm run seed`. Start the frontend and backend together with `npm run dev`. Open `http://localhost:5173` in the browser. The backend runs on `http://localhost:3001` and the frontend proxies `/api` requests to it during development.

Demo accounts

- Buyer: `buyer@campuscloset.test` / `password123`
- Seller: `seller@campuscloset.test` / `password123`
- Admin: `admin@campuscloset.test` / `password123`

Database restore

The database is a SQLite file at `code/database/campus_closet.sqlite`. To restore a clean demo state, run `npm run seed`. That command recreates the SQLite file with seeded users, active listings, a draft listing, a removed listing, a starter cart, wishlist items, notifications and a sample historical order

12. Closing Notes

Campus Closet meets the CPTS 489 Stack Requirement and carries every mid-submission use case into a working implementation with the exception of the two messaging use cases that were deliberately deferred. The application runs locally with a single install and a single seed command. The backend routes, the Sequelize models and the React pages all line up with the documentation in this report.